

Zero Boundary Algebra 1.1

A Typed State Calculus for Directional Boundaries, Mirror Symmetry, and Tamper-Evident Provenance

Shafaet Brady Hussain

Independent researcher; creator of Zero Boundary Algebra and architect of ZMath

Nottingham, United Kingdom

Contact: shaf@talktoai.org

ORCID: not asserted

Version 1.1.0 - 18 August 2026

Abstract

This paper defines Zero Boundary Algebra (ZBA) as a typed state calculus for representing entry into a boundary, an at-boundary reset, emergence from a boundary, polarity reversal, recursive verification, and provenance-preserving composition. The motivating notation uses the symbols -0 , 0 , and $+0$ for three operationally distinct boundary phases; 3 and 6 as human-readable positive and negative polarity labels; $9/0$ as completion or reset; and 8 as a mnemonic for an involutive mirror because $8 \equiv -1 \pmod{9}$. These symbols are labels and operators in a transition system, not claims that ordinary real arithmetic has been replaced.

The formal object is a many-sorted algebra $Z = (S, 0, A)$ whose states bind an asset identifier, polarity, boundary phase, recursion depth, policy, lineage, and cryptographic commitment. We specify typing rules, transition preconditions, an involutive mirror, an idempotent reset projection, an append-only evidence relation, and a distinction between exact state equality and audit equivalence. We prove core structural properties, identify the circumstances under which mirror and reset commute, and show how non-commutativity may encode history rather than contradiction. A reference record format, conformance levels, threat model, test vectors, and falsifiable evaluation programme are provided.

ZBA is intentionally positioned as an algebraic control and provenance framework under formal development. It does not itself encrypt data, establish cryptographic strength, provide post-quantum security, create quantum key distribution, prove legal ownership, or replace established provenance standards. Its practical role is to supply a compact, machine-checkable vocabulary around conventional authenticated encryption, signatures, hash chains, append-only logs, and interoperable provenance models. Version 1.1 adds an executable verifier: 432/432 finite structural property instances passed, one six-record chain was accepted, and 14/14 seeded field and topology faults were rejected. It also audits nine implemented encryption/protection profiles in the wider ZMath corpus while distinguishing original compositions from established primitives and third-party protocols.

Keywords: algebraic specification; labelled transition system; provenance; tamper evidence; directional zero; signed zero; state machine; authenticated metadata; audit log; ZMath.

1. Contribution and scope

Zero Boundary Algebra began as a symbolic vocabulary distributed across research notes and implementations. This paper turns that vocabulary into one testable object. Its principal contributions are:

1. a typed state space separating polarity, boundary phase, and evidence status;
2. explicit mirror, enter, seal/reset, emerge, verify, compose, and recurse operators;
3. axioms and derived propositions with declared limits;
4. a cryptographic binding profile for state metadata;
5. a mapping to W3C PROV concepts rather than a competing provenance ontology;
6. conformance levels that separate notation, transition validation, and cryptographic evidence;
7. a reproducible evaluation plan capable of rejecting the claim that ZBA improves audit work;
8. an executable reference verifier, deterministic vectors, and exhaustive finite-domain property checks;
9. a privacy-minimising disclosure model and deployment governance profile.

The paper makes no claim that the numerals 3, 6, 8, or 9 possess supernatural, physical, or cryptographic power. Their role is mnemonic. Security arises from reviewed cryptographic primitives and correct implementations. Audit value, if any, arises from explicit state semantics, transition constraints, and evidence binding.

2. Design requirements

The calculus is designed around eight requirements.

R1 - Direction matters. Arrival at a boundary and departure from it must be distinguishable even when a scalar projection maps both to zero.

R2 - State and evidence differ. A claimed state is not a verified state. Evidence status is represented separately.

R3 - Operations are typed. Ill-formed transitions fail closed instead of silently coercing values.

R4 - History is explicit. When an operation changes lineage, exact equality and audit equivalence are not conflated.

R5 - Metadata is bound. Security-relevant labels are authenticated or signed; decorative labels alone have no integrity force.

R6 - Conventional standards remain authoritative. ZBA is a domain-specific control layer, not a new cipher, hash, signature scheme, or provenance interchange standard.

R7 - Claims are falsifiable. Human and machine evaluations must compare ZBA against ordinary labelled-state baselines.

R8 - Private implementation is unnecessary for public verification. A verifier can validate a public record without private keys, plaintext, or proprietary application code, provided the relevant public keys, commitments, and policies are available.

3. Mathematical preliminaries

3.1 Sorts

Let the following carrier sets be non-empty:

- A: asset identifiers;
- $P = \{-, 0, +\}$: polarity labels;
- $B = \{-0, 0, +0\}$: boundary phases, read as entering, at-boundary, and emerging;
- N: non-negative recursion depths;
- Q: policy identifiers;
- H: fixed-length cryptographic commitments;
- L: finite event lineages;
- $E = \{\text{claimed}, \text{checked}, \text{sealed}, \text{rejected}\}$: evidence status;
- T: monotonically ordered logical times or sequence numbers.

A ZBA state is the tuple

$$s = (a, p, b, r, q, l, h, e) \text{ in } S = A \times P \times B \times N \times Q \times L \times H \times E.$$

The projections are written $\text{asset}(s)$, $\text{pol}(s)$, $\text{phase}(s)$, $\text{depth}(s)$, $\text{policy}(s)$, $\text{lineage}(s)$, $\text{commit}(s)$, and $\text{evidence}(s)$.

3.2 Directional zero is a tagged control value

The three values in B are not three real numbers. They are tagged control states. Define an erasure map

$$\text{epsilon} : B \rightarrow \{0\} \text{ with } \text{epsilon}(-0) = \text{epsilon}(0) = \text{epsilon}(+0) = 0.$$

Thus the tags are distinct in the control domain while sharing the same erased scalar value:

$$-0 \neq 0 \neq +0 \text{ in } B, \text{ but } \text{epsilon}(-0) = \text{epsilon}(0) = \text{epsilon}(+0).$$

This is analogous only in spirit to signed-zero representations in floating-point systems. ZBA attaches workflow meaning that IEEE floating-point arithmetic does not supply. Implementations must not serialize phase tags as bare JSON numbers: common canonicalization rules can collapse negative zero to zero. A conforming record serializes them as strings: "entering", "boundary", and "emerging".

3.3 Mnemonic numerals

Define a display map μ :

- $\mu(+) = 3$;
- $\mu(-) = 6$;
- $\mu(0) = 9/0$;
- $\mu(\text{mirror}) = 8$.

The mirror mnemonic follows $8 \equiv -1 \pmod{9}$, so multiplication by 8 reverses non-zero residues modulo 9 and is involutive because $8^2 \equiv 1 \pmod{9}$. This modular fact motivates the notation but does not by itself prove properties of a software system. Those properties follow only from the definitions below.

4. Signature and operators

Let 0 contain the following partial or total operations.

4.1 Polarity mirror

Define $\text{negP}(+) = -$, $\text{negP}(-) = +$, and $\text{negP}(0) = 0$.

The structural mirror $M : S \rightarrow S$ is

$$M(a, p, b, r, q, l, h, e) = (a, \text{negP}(p), b, r, q, l, h, e).$$

The evidence-preserving implementation operator M^* may additionally append an event and recompute the commitment. Accordingly, M models the algebraic effect and M^* models an audited execution.

4.2 Boundary transitions

For an operation identifier op , actor u , and logical time t , define partial transitions:

- $\text{Enter_op}(s)$ is defined when $\text{phase}(s) = +0$ or the asset is newly created; its result has phase -0 .
- $\text{Seal_op}(s)$ is defined when $\text{phase}(s) = -0$; its result has phase 0 and evidence at least checked.
- $\text{Emerge_op}(s)$ is defined when $\text{phase}(s) = 0$ and the policy's release predicate holds; its result has phase $+0$.
- $\text{Reject_op}(s)$ is defined for any unaccepted verification; its result has evidence rejected and cannot emerge without a new authorised recovery event.

The ordinary successful cycle is therefore

$$+0 \rightarrow -0 \rightarrow 0 \rightarrow +0.$$

The transition labels, not the phase tags alone, state what occurred: create, encrypt, rekey, export, recover, verify, revoke, or another policy-defined operation.

4.3 Reset projection

The structural reset $R : S \rightarrow S$ projects any permitted state to its at-boundary form:

$$R(a, p, b, r, q, l, h, e) = (a, 0, 0, r, q, l, h, e'),$$

where e' is checked if validation succeeds and rejected otherwise. In an audited system, R^* appends a seal event and recomputes h .

4.4 Recursive verification

The recursion operator $V : S \rightarrow S$ increments the verification depth after a successful verification:

$$V(a, p, b, r, q, l, h, e) = (a, p, b, r+1, q, l, h, \text{sealed}).$$

This operator does not mean mathematical infinity. Recursion is a finite, observable repetition count. Systems may impose a maximum depth and must reject wraparound.

4.5 Composition

For transition operators f and g , composition $g \circ f$ is defined only when $f(s)$ is defined and satisfies the preconditions of g . A composition carries the concatenated event sequence. Parallel asset histories form a product state; they do not implicitly share keys, authority, or evidence.

4.6 Audit equivalence

Define exact equality = componentwise over S . Define audit equivalence $\sim_a$ relative to an observer authorised for asset a :

$s \sim_a t$ iff their asset, policy, semantic payload commitment, and externally visible control state are equal, and any lineage difference consists only of valid, non-destructive audit events permitted by policy.

Audit equivalence is not cryptographic equality. It permits two states to describe the same controlled asset after reversible checks while retaining different evidence histories.

5. Axioms

The algebra adopts the following axioms.

A1 - Sort preservation. Every defined operator returns a state in S ; parsers reject unknown mandatory fields, invalid tags, depth underflow/overflow, and malformed commitments.

A2 - Asset invariance. Structural control operators do not change $\text{asset}(s)$.

A3 - Mirror involution. $M(M(s)) = s$.

A4 - Reset idempotence. $R(R(s)) = R(s)$ for states whose validation outcome is stable.

A5 - Phase order. A successful controlled operation follows $+0 \rightarrow -0 \rightarrow 0 \rightarrow +0$; shortcuts require a separately named exception transition.

A6 - Evidence monotonicity. Within an uncompromised lineage, a record cannot silently move from rejected to sealed, nor delete an event. Recovery creates a new event and preserves the rejected predecessor.

A7 - Commitment recurrence. For canonical event bytes $C(E_n)$, previous commitment $h_{(n-1)}$, state projection $C(S_n)$, and domain separator D ,

$$h_n = H(D \parallel h_{(n-1)} \parallel C(E_n) \parallel C(S_n)).$$

A8 - Authorisation. A transition is valid only if the actor, policy version, operation, and preconditions are authorised and the relevant signature or MAC verifies.

A9 - Nonce discipline. If an AEAD profile is used, its nonce uniqueness and key-management requirements are independent invariants; ZBA labels cannot repair nonce reuse.

A10 - Verifier determinism. Given the same canonical record, policy, trust anchors, and validation time, conforming verifiers return the same result.

6. Propositions and proofs

Proposition 1 - Mirror is an involution

For every s in S , $M^2(s) = s$.

Proof. M changes only the polarity component using negP . The map swaps $+$ and $-$ and fixes 0 , so $\text{negP}(\text{negP}(p)) = p$. Every other component is unchanged. Therefore applying M twice returns every component of s . QED.

For the audited implementation M^* , exact equality usually fails because two events have been appended. The appropriate invariant is $M^*(M^*(s)) \sim_a s$, provided both mirror events verify and the policy treats them as semantically reversible.

Proposition 2 - Structural reset is idempotent

For every state with stable validation outcome, $R^2(s) = R(s)$.

Proof. One application sets polarity and phase to their boundary values and chooses a deterministic evidence result. A second application makes the same assignments. All other structural components are fixed. QED.

The audited operator R^* need not be exactly idempotent because repeated sealing attempts may append distinct observations. Conforming systems either reject a redundant seal or classify it as a no-op observation.

Proposition 3 - Mirror and reset commute structurally

For the definitions above, $R(M(s)) = M(R(s))$.

Proof. R sets polarity to $\emptyset$, which M fixes. Both sides set the phase to $\emptyset$ and preserve the other structural components. QED.

This proposition is deliberately narrow. Audited implementations may not commute: $R^* \circ M^*$ and $M^* \circ R^*$ append different ordered events and therefore produce different commitments. They can be semantically related while remaining cryptographically distinct. This ordered difference is useful because it records whether reversal occurred before or after sealing.

Proposition 4 - Single-event tampering propagates

Assume collision resistance of H , unambiguous canonicalization, and a trusted anchor for $h_{\emptyset}$. If any committed event or state component at index i is modified without recomputing and reauthorising all later records, verification fails at i or a later anchor.

Proof sketch. By A7, h_i depends on the previous commitment and canonical bytes at i ; every later h_j depends recursively on h_i . A changed input changes the digest except with a collision or preimage-style failure. If an attacker recomputes later hashes, signatures or external anchors no longer match unless the attacker also controls the corresponding authority. QED under the stated assumptions.

Proposition 5 - A label without binding supplies no tamper evidence

If phase, polarity, or policy is excluded from the authenticated data, signature payload, and hash-chain input, an attacker capable of editing the record may relabel it without invalidating cryptographic checks.

Proof. Verification is a function only of bound inputs. Changing an unbound field does not change that function's input, so the check result is unchanged. QED.

This proposition is the central engineering constraint: symbolic provenance becomes security-relevant only when it is cryptographically bound.

7. Canonical record and cryptographic binding profile

7.1 Record fields

A minimum public record contains:

- spec: fixed identifier and version;
- asset_id: opaque stable identifier;
- sequence: unsigned monotonically increasing integer;
- operation: policy-defined transition label;
- polarity: positive, neutral, or negative;
- phase: entering, boundary, or emerging;

- `recursion_depth`: unsigned integer;
- `policy_id` and `policy_version`;
- `previous_commitment`;
- `payload_commitment`;
- `event_time` or a declared logical clock;
- `actor_id` or privacy-preserving actor reference;
- `evidence_status`;
- `commitment_algorithm` and `commitment`;
- optional signature suite, public-key reference, and signature;
- optional external anchor reference.

7.2 Canonicalization

Interoperable hashing requires one byte representation. The reference profile uses UTF-8 JSON canonicalized according to RFC 8785, but phase and polarity values are strings. Unknown optional fields may be preserved; unknown critical fields cause rejection. Domain-separation strings distinguish event commitments, payload commitments, and signature inputs.

7.3 Authenticated encryption

Where the record accompanies an encrypted payload, the ZBA control projection should be included as authenticated additional data (AAD) or in a signed manifest. NIST SP 800-38D defines GCM as authenticated encryption with associated data: AAD is authenticated but not encrypted. A ZBA implementation therefore may keep non-sensitive control metadata visible while making unauthorised changes detectable. Visibility can leak workflow information, so sensitive metadata should instead be encrypted or tokenised.

Two layers of AES-GCM do not constitute AES-512 and do not automatically double security. If a system uses two layers, the defensible rationale is factor separation, compartmentalisation, or defence in depth, subject to independent keys, correct nonces, and a reviewed composition.

7.4 Public-key and post-quantum boundary

Classical ephemeral ECDH is not post-quantum secure against a sufficiently capable cryptographic quantum computer. A future hybrid profile may combine a classical agreement with a standardised post-quantum KEM such as ML-KEM, defined in NIST FIPS 203. Merely recording a quantum-computer job identifier does not provide post-quantum confidentiality.

7.5 External anchors

An external execution record can bind a provider job ID, circuit fingerprint, backend, status, result reference, and the ZBA commitment. This is an evidence anchor, not a quantum encryption primitive and not QKD. Its value depends on authenticity, availability, uniqueness, binding, and the independence of the external record. Append-only Merkle logs or trusted timestamps are alternative anchor mechanisms.

8. Mapping to established provenance concepts

ZBA is a domain-specific state vocabulary. W3C PROV remains the interoperability layer. A recommended mapping is:

ZBA concept	W3C PROV concept	Qualification
protected asset/version	prov:Entity	Each materially changed version is a new entity.
encrypt, rekey, verify, export	prov:Activity	Transition event with start/end or logical sequence.
operator, device, service	prov:Agent	Responsibility and attribution are policy-dependent.
input to operation	prov:used	The activity used the preceding entity.
output version	prov:wasGeneratedBy	The successor entity was generated by the activity.
lineage edge	prov:wasDerivedFrom	Does not by itself prove integrity.
author attribution	prov:wasAttributedTo	Attribution is not identical to legal ownership.
policy	prov:Plan	The plan describes permitted transitions.
ZBA evidence bundle	prov:Bundle	A named set of provenance descriptions.

ZBA adds phase, polarity, transition guards, and commitment recurrence. It should export rather than replace PROV-compatible statements.

9. Threat model

9.1 Protected properties

The model seeks to support integrity of transition metadata, detection of broken lineage, explicit policy/version binding, reproducible verification, and separation of claimed from checked evidence.

9.2 Adversaries

Consider an adversary who may read or alter stored records, reorder or delete events, replay an old valid record, relabel a state, substitute a payload, exploit parser differences, compromise an endpoint, steal an application credential, or present an unrelated external job as an anchor.

9.3 Required mitigations

- hash chaining or an append-only authenticated structure;
- signatures or MACs with explicit trust anchors;
- sequence and freshness policy;
- canonical serialization and critical-field handling;
- binding of asset, operation, phase, polarity, policy, payload commitment, and predecessor;
- domain separation;
- rollback detection using an independent checkpoint or witness;
- endpoint hardening and key lifecycle controls;
- privacy review for visible metadata;
- recovery procedures that append rather than erase failures.

9.4 Out of scope

ZBA alone does not prevent endpoint compromise, key theft, traffic analysis, denial of service, coercion, malicious authorised actors, cryptographic implementation bugs, long-term algorithm failure, or false source data entering an otherwise intact log. Provenance can show what a system recorded; it cannot guarantee that the recorded event was truthful without trustworthy sensing and authority.

10. Conformance levels

Level 0 - Notation. Uses the vocabulary but makes no machine-verifiable integrity claim.

Level 1 - Typed transitions. Validates the state schema, phase order, guards, policy versions, and deterministic error results.

Level 2 - Bound evidence. Implements canonical commitments over all critical fields and verifies predecessor continuity.

Level 3 - Authenticated evidence. Adds signatures or MACs, trust-anchor handling, algorithm identifiers, key status, and replay/rollback controls.

Level 4 - Witnessed evidence. Adds an independent trusted timestamp, transparency log, notarisation, or external execution anchor with a documented binding protocol.

A product must state its level and profile. It must not call a Level 0 decorative label tamper-evident.

11. Reference transition policy

The reference policy is fail-closed:

Current phase	Requested operation	Preconditions	Next phase	Failure
emerging/new	enter operation	actor and operation authorised	entering	reject event
entering	seal/reset	payload and policy checks pass	boundary	rejected boundary
boundary	emerge/release	release predicate and signature pass	emerging	remain boundary/rejected
any	verify	chain, signatures, policy and payload commitment pass	unchanged; depth + 1	evidence rejected
any	recover	recovery authority and predecessor preserved	entering	reject event

An emergency transition may exist, but it must be a named policy operation with a reason code, elevated authorisation, and subsequent review. It must never be disguised as a normal cycle.

12. Test vectors and verifier obligations

The accompanying reference artifacts define deterministic positive and negative cases. At minimum, a verifier must test:

1. a valid create-enter-seal-emerge sequence;
2. mirror twice, demonstrating structural equality and distinct audited commitments;
3. reset twice, demonstrating structural idempotence or a declared no-op event;
4. mutation of phase, polarity, operation, policy, predecessor, payload commitment, and depth;
5. deletion, insertion, duplication, and reordering of events;

6. wrong signature, unknown key, revoked key, and unsupported critical field;
7. rollback to an earlier valid prefix when an external checkpoint exists;
8. JSON -0, duplicate keys, invalid Unicode, oversized integers, and non-canonical encodings;
9. incorrect AEAD nonce reuse detection at the surrounding cryptographic layer;
10. external anchor mismatch between job reference and committed record.

The verifier must return a structured result with the first failing sequence, failure class, critical field, and policy version. Boolean-only results are insufficient for audit use.

12.1 Executable reference result

Version 1.1 accompanies the prose specification with an executable Python reference profile. The program is intentionally small enough to review: it defines an immutable structural state, mirror and reset functions, a restricted canonical JSON encoding, SHA-256 commitment recurrence, shape validation, transition validation, and chain verification. It does not implement signatures, AEAD, trusted time, key storage, revocation services, or external witnesses, so passing its tests is Level 2 evidence rather than a product security assessment.

The finite-domain checker enumerates every tuple in $P \times B \times \{0, 1, 2, 3\} \times E$, giving $3 \times 3 \times 4 \times 4 = 144$ structural states. It checks mirror involution, reset idempotence, and structural mirror/reset commutation over every state. All 432 property instances passed: 144/144 for each property. A deterministic six-record chain representing enter, seal, emerge, recursive verify-enter, verify-seal, and verify-emerge was accepted.

Fourteen negative cases were then constructed by changing every major committed field or modifying chain topology. Mutations of phase, polarity, operation, policy version, predecessor, payload commitment, recursion depth, evidence status, asset identifier, sequence, and commitment were rejected. Deletion, reordering, and duplication were also rejected. The observed result was 14/14 seeded faults detected.

These figures establish conformance of the executable model to the stated finite properties and supplied fault set. They do not demonstrate cryptographic security, field completeness outside the profile, user benefit, collision resistance by experiment, or resistance to an attacker holding a valid signing key.

12.2 Reproducibility record

The release contains `zba_reference.py` for executable semantics, `property_check.py` for finite-domain and mutation testing, `test-vectors.json` for the deterministic chain, `property-check-results.json` for observed output, `zba.schema.json` for structure, and `SHA256SUMS.txt` for release digests.

A rerun uses Python 3.11 or later and executes `python property_check.py` from the artifact directory. Independent implementations should ingest the same positive vector and reproduce every negative result. A future suite should add signatures, key rotation, canonicalization edge cases, rollback witnesses, parser differentials, and cross-language vectors.

12.3 Complexity and storage

For a linear chain of n records, verification is $O(n)$ in record count plus the cost of reading and hashing committed bytes. Each record can be streamed while retaining only the previous verified record, trust state, and policy, so working memory is $O(1)$ aside from record and policy size. Random access to a late record without checkpoints still requires predecessor validation.

The linear profile favours simple custody histories and portable files. A Merkle or transparency-log profile is preferable when many independent records must be included or auditors need compact proofs. ZBA does

not prescribe one authenticated data structure for every deployment; it prescribes semantic fields and requires ordering and critical metadata to be bound.

13. Operational semantics

13.1 Judgements

Let Γ be a verification environment containing a policy registry, trust anchors, algorithm policy, logical clock, and external checkpoints. Write $\Gamma \vdash s \xrightarrow{\text{op}/u, t} s'$ when actor u at logical time t is authorised by Γ to apply op to s , producing s' . A derivation is accepted only when its premises hold.

ENTER. $\text{phase}(s) = \text{emerging}$, actor and operation authorisation, $\text{phase}(s') = \text{entering}$, and $\text{prev}(s') = \text{commit}(s)$ permit an enter derivation.

SEAL. $\text{phase}(s) = \text{entering}$, successful mandatory checks, $\text{phase}(s') = \text{boundary}$, and $\text{evidence}(s') \geq \text{checked}$ permit a seal derivation.

EMERGE. $\text{phase}(s) = \text{boundary}$, a true release predicate, $\text{phase}(s') = \text{emerging}$, and authentic successor evidence permit emergence.

REJECT. If a mandatory premise fails, the successful judgement is not derivable. An implementation may append a rejected observation, but it cannot manufacture the requested successor.

RECOVER. A rejected predecessor, authorised recovery role, non-empty reason, entering successor, and exact predecessor commitment permit a separately labelled recovery derivation. Recovery preserves the rejected predecessor.

These rules make failure absence meaningful: a verifier rejects any state change for which it cannot build a derivation.

13.2 Safety properties

The transition system aims to preserve six safety properties.

- **No silent shortcut:** an ordinary cycle cannot jump between non-adjacent phases.
- **No orphan successor:** every non-genesis linear record declares exactly one matching predecessor.
- **No invisible relabelling:** every security-relevant phase, polarity, operation, policy, and payload reference is committed.
- **No silent resurrection:** rejected evidence requires a recovery transition before successful re-entry.
- **No ambiguous critical extension:** an unknown critical field causes rejection.
- **No scalar-zero coercion:** boundary phases remain enumerated tags through parsing, storage, hashing, and display.

13.3 Liveness assumptions

Safety does not guarantee progress. Emergence may remain blocked forever if approval never arrives, a key is unavailable, a witness is offline, or policy is inconsistent. A deployment claiming liveness must define timeouts, quorum rules, degraded modes, and recovery authority. Any degraded transition remains distinguishable in the history.

13.4 Determinism and policy time

Verifier determinism is relative to a fixed environment. Revocation and policy change introduce time. A record may have been valid when created but unacceptable later. Results should therefore distinguish `valid_at_event_time`, `valid_under_current_policy`, `cryptographically_intact`,

externally_witnessed, and semantically_authorised. Collapsing these dimensions into one word such as valid creates false assurance.

14. Relationship to neighbouring formalisms

14.1 Labelled transition systems and finite-state machines

ZBA is closest to a guarded labelled transition system. Conventional state machines already represent states, events, guards, and actions. ZBA's candidate contribution is not the discovery of state machines; it is a specialised product state combining a three-phase directional boundary tag, polarity involution, recursive evidence depth, policy identity, and cryptographic lineage. The B1 evaluation baseline is deliberately a conventional typed state machine so the added notation must earn its place.

14.2 Process algebra

Process algebras study composition, concurrency, communication, equivalence, and ordering more deeply than this specification. ZBA 1.1 defines sequential composition and product histories but not a full calculus of synchronisation, hiding, choice, or bisimulation. Future work should define trace, observational, and refinement relations. Until then, audit equivalence is a domain relation, not a substitute for established process-equivalence theory.

14.3 Event sourcing

Event-sourced systems derive current state from append-only history. ZBA can be implemented as an event-sourced aggregate, but it adds normative phase guards and commitment recurrence. Mature event sourcing adds snapshotting, projections, migration, idempotent consumers, and distributed consistency that ZBA does not solve. The two are complementary.

14.4 Temporal logic and model checking

The phase cycle supports temporal properties such as `always(rejected -> not sealed until recover)`, `always(non_genesis -> has_predecessor)`, and `always(emerge -> previously boundary)`. TLA+, Alloy, Lean, or Coq would provide stronger assurance than the finite Python checker. The executable model is an accessible bridge, not the final mechanised-verification layer.

14.5 W3C PROV and attestations

W3C PROV supplies interoperable entities, activities, agents, derivations, attribution, and bundles. Software supply-chain frameworks supply build attestations. ZBA should specialise or export to these ecosystems: phase and polarity can be qualified attributes, a transition is an activity, policy is a plan, and a signed record can be an attestation predicate. A closed vocabulary unable to map outward would reduce auditability.

14.6 Signed-zero notation collision

IEEE 754 signed zero is numerical and has defined arithmetic behaviour. ZBA directional zero is a tagged workflow phase. RFC 8785 canonicalization illustrates why they must not be conflated: numeric negative zero can serialize as zero. The wire values are therefore entering, boundary, and emerging.

15. Privacy and selective disclosure

15.1 Provenance can become surveillance

Records containing actors, times, operations, assets, locations, and external references can expose relationships even when payloads are encrypted. Fine-grained provenance may reveal work rhythms, organisational charts, project existence, incident timing, or protected-asset names. Metadata confidentiality is therefore a separate objective.

15.2 Data minimisation profile

A minimised public record uses opaque asset IDs, pseudonymous or role-based actor references, coarse or logical time where precision is unnecessary, non-revealing policy IDs, and commitments rather than descriptions. It omits device fingerprints, IP addresses, personal names, and file paths unless purpose requires them. The rule is to publish only what the intended verifier needs for the intended claim.

15.3 Commitment limitations

Hashing low-entropy metadata does not hide it: an observer can enumerate likely values. Salting resists simple enumeration but complicates public verification; keyed commitments conceal values but introduce key distribution. Each profile must state whether a field is public, encrypted, salted, keyed, selectively disclosed, or omitted.

15.4 Redaction and erasure tension

Append-only evidence can conflict with deletion obligations. A robust design avoids personal data in immutable logs, stores revocable encrypted references, and defines how tombstones, key destruction, and legal holds affect verification. A deletion event may prove retirement without retaining underlying data. Hashing alone does not solve governance.

15.5 Selective disclosure roadmap

Future profiles may use Merkle commitments, verifiable credentials, or zero-knowledge proofs to disclose selected fields or predicates. These add complexity and require explicit threat models. They must not be inferred merely from the name Zero Boundary.

16. Governance and lifecycle

16.1 Versioning

Version 1.1 uses semantic versioning at document level. A patch clarifies without changing accepted records. A minor version adds backward-compatible fields or profiles. A major version may change semantics, rules, or canonicalization. Every release retains prior versions, digests, migration notes, and schemas.

16.2 Algorithm agility

Algorithm identifiers are committed data. Trusted policy specifies allowed hash, signature, KDF, KEM, and AEAD suites; activation and retirement; sizes; transitions; and downgrade handling. A verifier never accepts an algorithm merely because a record requests it. Crypto-agility also needs evidence of which policy authorised the suite and whether the key was active.

16.3 Key compromise

If a signing key is compromised, hash continuity may remain while authorship assurance changes. The evidence service appends a compromise or revocation statement, identifies the affected interval, preserves independent witnesses, and re-anchors unaffected material where possible. Rewriting history destroys investigative evidence.

16.4 Clock and ordering failures

Wall clocks drift and can be manipulated. The reference profile prioritises per-asset sequence numbers and treats timestamps as context. Distributed deployments may need vector or Lamport clocks, consensus order, or trusted timestamps. A timestamp alone is not proof of order or existence.

16.5 Separation of duties

High-assurance deployments distinguish creator, operator, approver, recovery authority, policy administrator, key custodian, verifier, and witness. Small systems may combine roles, but evidence should reveal that combination rather than imply independence.

16.6 Incident response

Runbooks specify how to freeze emergence, preserve rejected evidence, rotate keys, record emergency transitions, validate checkpoints, restore backup, and communicate uncertainty. The calculus supplies labels; readiness depends on people, rehearsals, monitoring, and recovery systems.

17. Worked example

Consider a protected manuscript with opaque asset ID `urn:zba:paper:001`. Its initial digest becomes the payload commitment. Creation enters the boundary with positive polarity and claimed evidence. Sealing verifies required metadata and produces a neutral boundary record. Authorised release emerges with positive polarity and sealed evidence. Later verification repeats the cycle at recursion depth one.

The six released vectors implement this history. Changing record two's policy without recomputing its commitment fails at record two. Recomputing without an authorised signature would fail at Level 3. Deleting record two breaks record three's sequence and predecessor. Rolling the entire chain back requires an independent Level 4 checkpoint to prove that later records existed.

This separates four claims: the bytes form a continuous history; authorised keys authenticated it; an independent witness observed it; and the real-world assertions were true. ZBA can represent the first three. The fourth still depends on trustworthy people, devices, measurements, and procedures.

18. Evaluation programme

18.1 Research questions

- RQ1: Does ZBA reduce the time reviewers need to reconstruct an asset's transition history?
- RQ2: Does it improve detection of contradictory, missing, reordered, or unauthorised states?
- RQ3: Does the mnemonic notation add value beyond an ordinary finite-state-machine vocabulary?
- RQ4: Can independent implementations produce identical verification results?
- RQ5: What privacy and cognitive costs arise from the additional metadata?

18.2 Baselines

Compare three conditions:

- B0: ordinary timestamped application logs;
- B1: a conventional typed provenance state machine with descriptive labels;
- B2: the same state machine with ZBA directional-phase and mirror vocabulary.

B1 is essential. Comparing only against unstructured logs would not isolate whether ZBA's distinctive notation adds value.

18.3 Tasks and measures

Construct blinded record sets with valid histories and seeded defects. Participants answer lineage, authority, contradiction, and recovery questions. Primary outcomes are defect-detection F1, time to correct reconstruction, inter-rater agreement, and false assurance rate. Secondary outcomes are training time, perceived workload, terminology recall, verifier interoperability, record-size overhead, and privacy leakage

identified by reviewers.

18.4 Hypotheses and rejection criteria

- H1: B2 improves defect-detection F1 over B0.
- H2: B2 is non-inferior to B1 on accuracy and faster on boundary reconstruction.
- H3: independent Level 2 verifiers agree on every normative vector.

If B2 does not outperform or reach non-inferiority against B1, the distinctive ZBA notation has not demonstrated operational value even if the underlying state machine remains sound. If independent verifiers disagree, the specification is ambiguous and must be revised. If users systematically confuse evidence status with cryptographic assurance, the vocabulary must be redesigned.

18.5 Security evaluation

Use property-based testing for phase transitions and mirror/reset laws; mutation testing for every critical field; parser differential testing; fuzzing of canonical records; signature and trust-store negative tests; AEAD misuse checks; rollback simulations; and an independent threat-model review. Public results must distinguish specification conformance from a full product security audit.

19. Implementation evidence and boundaries

The wider ZMath ecosystem provides motivating implementations: browser-side protected artifacts, encrypted messaging workflows, portable containers, provenance records, and optional external execution evidence. These show that the vocabulary has been applied across multiple software surfaces. They do not constitute independent validation of the algebra, cryptographic review, quantum advantage, or formal verification.

For Zmail-style dual-input protection, the public claim should remain precise: separate passphrase and visual-pattern inputs may derive independent key material using separate salts and standard KDFs, followed by authenticated encryption. The security contribution is factor separation and implementation discipline, not a multiplication of AES key length. Ordinary email metadata and unsupported delivery paths can remain outside end-to-end protection and must be disclosed.

For QuantumEncryption1-style evidence, provider job IDs and circuit fingerprints can serve as external provenance references when cryptographically bound. They must not be described as secretly generating encryption keys unless a documented key protocol actually does so, and they are not evidence of QKD.

20. Encryption-system audit

20.1 Classification rule

The author's software corpus contains more than five implemented protection profiles, but the scientifically correct claim is not "five newly invented encryption algorithms." AES-GCM, PBKDF2, HKDF, HMAC-SHA-256, P-256 ECDH, P-256 ECDSA, and Matrix Olm/Megolm are established primitives or protocols. The original work lies in the factor combination, container/envelope formats, policy modes, product integration, provenance binding, and operational architecture.

This audit distinguishes:

- a **primitive**, such as AES-GCM or HMAC;
- a **construction**, which composes primitives into confidentiality and integrity behaviour;
- a **format/profile**, which defines fields, factor derivation, AAD, and failure rules;

- a **protocol**, which distributes or wraps keys between devices;
- an **integration**, which deploys an established third-party protocol in the ecosystem.

20.2 Confirmed implemented profiles

ID	Implemented protection profile	Main mechanism	Authorship classification	Evidence boundary
E1	ZME1 Portable	Password PBKDF2-SHA-256 plus pattern-image SHA-256, ZEQ-labelled mixing, HKDF-SHA-256, AES-256-GCM with authenticated metadata	Original container and factor-composition profile using standard primitives	One AEAD encryption layer; not a new AES variant
E2	ZME1 Exclusive	E1 plus a server-supplied policy lock bound into derivation	Original policy-bound variant	Availability and trust depend on policy endpoint
E3	ZMath Shield .zmath vault	Separate passphrase and visual-pattern derivations with independent salts; two AES-256-GCM layers	Original dual-factor, dual-layer protected-artifact profile using standard primitives	Two layers do not equal AES-512
E4	CallChat ZShield/ZSHIELD1	PBKDF2-SHA-256 plus HKDF-SHA-256 mixed factors, AES-256-GCM, canonical authenticated header, message and file envelopes	Original application profile and envelope integration	Often transported inside Matrix E2EE; layers have different roles
E5	Zmail protected mail	Two fresh AES-256-GCM content layers; per-device ephemeral P-256 ECDH and HKDF key wrapping; P-256 ECDSA sender signature; opaque recipient slots	Original protected-mail envelope protocol using standard primitives	Classical, not post-quantum; applies only to ready local recipients
E6	ZNotes encrypted store	Dual AES-256-GCM server-side encrypted storage with per-user HMAC isolation	Application storage construction	Service can decrypt; this is encrypted at rest, not end-to-end
E7	ZeroThink .ztz / Z-Pepper	PBKDF2-HMAC-SHA-256 derives encryption and MAC keys; HMAC-SHA-256 counter stream; encrypt-then-MAC-style envelope and plaintext digest check	Custom legacy construction and file format	Requires independent cryptographic review; prefer standard AEAD for new designs
E8	Matrix Olm/Megolm rooms	Matrix end-to-end encryption implemented by the Matrix client stack	Third-party protocol integrated into CallChat	Not authored as a new cipher by the ZBA creator
E9	Local auto-shield records	Random AES-256-GCM key and authenticated local record/AAD	Original local application integration	Separate from user-factor ZME1 derivation

The count is therefore nine identifiable profiles/integrations in the reviewed corpus, seven of which represent author-created constructions, formats, or integrations around standard cryptography. Counting can vary depending on whether Portable and Exclusive, or local and messaging envelopes, are grouped.

The most defensible public wording is:

> The ZMath ecosystem implements at least seven distinct author-created encryption or protected-data profiles, plus integrated Matrix end-to-end encryption, using established cryptographic primitives. These are original compositions, formats, and application protocols—not claims of seven newly invented base ciphers.

20.3 Executed verification on 18 August 2026

Targeted local test suites were rerun against the reviewed source snapshots. Zmail protected mail passed six tests covering dual-layer round trip, non-recipient refusal, tamper rejection, opaque recipient slots, sender-fingerprint forgery rejection, and fail-closed Roundcube synchronisation. ZMath Shield passed six tests covering note and attachment round trips through two AES-GCM layers, plaintext and metadata absence from exported JSON, wrong-factor and tamper failure, weak-input rejection, absence of network/persistent-storage primitives in the browser vault module, and visual-pattern controller behaviour.

CallChat ZShield passed its file/message round-trip, tamper, factor, and resource-bound tests. Its QPU-factor module passed derivation, evidence, tamper, and simulator-boundary tests. ZMath Auto passed its separate-factor recovery and local-processing contract. Three additional Zmail token tests passed for opaque session verification, tamper/wrong-key rejection, and purpose/expiry enforcement. These are implementation regression tests, not independent security audits.

20.4 Security interpretation

The strongest engineering contribution is architectural diversity with shared discipline: random salts and nonces, explicit KDF parameters, AAD-bound metadata, tamper rejection before plaintext release, separation of factors, per-device key wrapping, signatures, and documented boundaries. The primary research risk is vocabulary inflation. A profile name such as ZEQ-256 or QPU factor must not imply a novel reviewed cipher or quantum confidentiality.

The .ztz HMAC counter-stream construction deserves special caution. HMAC can be used to construct pseudorandom output, and the envelope authenticates ciphertext, but a bespoke stream construction carries nonce, domain-separation, misuse, and interoperability risk. New designs should prefer standard AEAD unless an independent cryptographic review establishes a reason and a proof-oriented specification for the custom mode.

The QPU-derived factor is additional derivation input and evidence, not entropy that should be trusted alone and not a replacement for a local CSPRNG. The implemented profile correctly combines local random material with the external measurement through HKDF. Public claims should describe it as a quantum-connected factor experiment, not quantum key distribution.

20.5 Relationship between the algebra and encryption profiles

ZBA does not become a cipher because it is used by encrypted products. Its role is to describe the lifecycle surrounding these profiles:

- **entering**: plaintext selection, factor capture, ephemeral-key generation, or rekey initiation;
- **boundary**: AEAD completion, signature verification, policy decision, or sealed container state;
- **emerging**: authorised delivery, export, recovery, or plaintext release;
- **mirror**: reversible audit comparison, sender/recipient perspective, or polarity check;
- **recursion**: re-verification, rotation, re-encryption, or external witnessing.

This separation strengthens both claims. The algebra supplies control semantics; conventional cryptography supplies confidentiality and authenticity; specific ZMath profiles supply original application constructions; external quantum jobs supply optional evidence references.

21. Limitations

This is a public formalisation by the framework's creator, not peer review or independent validation. The axioms are a design proposal. Some choices, especially audit equivalence and policy semantics, require domain-specific refinement. The paper supplies proof sketches and finite executable checks, not a mechanised proof-assistant development. No user study is reported. The 14 seeded negative cases are illustrative and are not an estimate of real-world detection rate. No claim of mathematical novelty relative to all prior labelled transition systems, process algebras, event sourcing, temporal logic, or provenance systems is made. The originality claim is narrower: the particular directional-boundary, polarity, mirror, recursive-evidence, and applied cryptographic-lifecycle synthesis named Zero Boundary Algebra, together with this specification.

Legal provenance is also limited. Hashes, signatures, timestamps, and public releases may strengthen an evidence trail, but they do not automatically establish patentability, copyright ownership, trade-secret status, contractual rights, authorship priority in every jurisdiction, or institutional endorsement.

22. Research roadmap

1. Publish the schema, vectors, verifier, and immutable release digest.
2. Invite mathematical, security, provenance, and human-factors review.
3. Mechanise the core transition system in TLA+, Alloy, Lean, or Coq.
4. Implement W3C PROV export and validate round-trip semantics.
5. Run the B0/B1/B2 blinded evaluation with preregistered metrics.
6. Commission an independent cryptographic design review of any ZMath profile claiming Level 3 or 4.
7. Define a hybrid post-quantum profile only after threat, interoperability, downgrade, and key-lifecycle analysis.
8. Revise the algebra based on counterexamples and verifier divergence.

23. Conclusion

Zero Boundary Algebra is presented here as a typed algebraic state calculus for directional reset phases, polarity mirror operations, recursive verification, and cryptographically bound provenance. It has an explicit carrier/state space, typed operators, axioms, derived laws, operational judgements, conformance levels, wire representation, and executable reference semantics. The formal mirror is involutive; structural reset is idempotent; the two commute structurally while audited executions preserve meaningful order. The v1.1 checker exhaustively confirmed these laws over a 144-state finite domain and rejected every supplied mutation. This is sufficient to present a concrete new named algebraic specification for review, while broader mathematical novelty and practical benefit remain questions for independent comparison and peer criticism.

The encryption audit shows why the algebra matters operationally without conflating it with cryptography: the reviewed ecosystem contains at least seven author-created encryption or protected-data profiles plus integrated Matrix E2EE, while confidentiality still comes from established primitives and implementation discipline. The next scientific question is empirical: whether reviewers and systems using ZBA detect defects more reliably than those using ordinary provenance state machines. Publishing the specification, code, vectors, boundaries, negative tests, and rejection criteria makes that claim criticisable and therefore researchable.

References

1. Hussain, S. B. (2026). *Creating a New Mathematical System 0,-0,+0*. Research Forum Online. <https://research.talktoai.org/research-papers/creating-a-new-mathematical-system/>
2. Hussain, S. B. (2026). *Zero Boundary Algebra as a Provenance Workflow for AI Research Notes*. ResearchForumOnline Research repository.
3. W3C. (2013). *PROV-O: The PROV Ontology*. W3C Recommendation. <https://www.w3.org/TR/prov-o/>
4. Moreau, L., & Missier, P. (eds.). (2013). *PROV-DM: The PROV Data Model*. W3C Recommendation. <https://www.w3.org/TR/prov-dm/>
5. Dworkin, M. (2007). *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST SP 800-38D. <https://doi.org/10.6028/NIST.SP.800-38D>
6. NIST. (2024). *Module-Lattice-Based Key-Encapsulation Mechanism Standard*. FIPS 203. <https://doi.org/10.6028/NIST.FIPS.203>
7. Rundgren, A., Jordan, B., & Erdtman, S. (2020). *JSON Canonicalization Scheme (JCS)*. RFC 8785. <https://doi.org/10.17487/RFC8785>
8. Laurie, B., Langley, A., & Kasper, E. (2013). *Certificate Transparency*. RFC 6962. <https://doi.org/10.17487/RFC6962>
9. NIST. *Hash chain*. Computer Security Resource Center Glossary. https://csrc.nist.gov/glossary/term/hash_chain
10. IEEE. (2019). *IEEE Standard for Floating-Point Arithmetic*. IEEE 754-2019. <https://standards.ieee.org/ieee/754/6210/>
11. QuantumEncryption1. (2026). *Zero Boundary Algebra for encrypted provenance and control*. <https://quantumencryption1.com/zero-boundary-algebra/>
12. QuantumEncryption1. (2026). *Technology architecture*. <https://quantumencryption1.com/technology/>
13. Krawczyk, H., & Eronen, P. (2010). *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*. RFC 5869. <https://doi.org/10.17487/RFC5869>
14. Krawczyk, H., Bellare, M., & Canetti, R. (1997). *HMAC: Keyed-Hashing for Message Authentication*. RFC 2104. <https://doi.org/10.17487/RFC2104>
15. Matrix.org Foundation. (2026). *Olm & Megolm*. Matrix Specification 1.18. <https://spec.matrix.org/v1.18/olm-megolm/>
16. Bradner, S. (1997). *Key words for use in RFCs to Indicate Requirement Levels*. RFC 2119; updated by RFC 8174. <https://doi.org/10.17487/RFC2119>

Appendix A - Normative invariants

1. $\text{asset}(s') = \text{asset}(s)$ for every structural control transition.
2. $M(M(s)) = s$.
3. $R(R(s)) = R(s)$ under stable validation.
4. $R(M(s)) = M(R(s))$ structurally.
5. Successful normal phases follow emerging $\rightarrow$ entering $\rightarrow$ boundary $\rightarrow$ emerging.
6. Every non-genesis record commits to exactly one predecessor within a single asset lineage.
7. Every critical field is included in the canonical commitment and signature/MAC input.

8. A rejected event is never silently overwritten.
9. Phase values are tagged strings, not signed numeric zero.
10. Verification is deterministic for fixed inputs and trust state.

Appendix B - Claim boundary

Statement	Status
ZBA defines a typed state calculus	Defined in this specification
Structural mirror is involutive	Proven from the definition
Structural reset is idempotent	Proven under stable validation
Hash-linked, authenticated records can expose mutation	Conditional on cryptographic and implementation assumptions
ZBA improves reviewer performance	Hypothesis; not yet demonstrated
ZMath products implement every conformance level	Not asserted
Two AES-GCM layers equal AES-512	Rejected
ECDH is post-quantum secure	Rejected
Quantum execution evidence is QKD or quantum encryption	Rejected
Publication proves legal ownership or institutional endorsement	Rejected
ZBA 1.1 presents a concrete new named algebraic specification	Supported by the carrier, operators, axioms, semantics, laws, and executable profile; broader novelty awaits independent review
The ecosystem contains more than five encryption/protection profiles	Supported by source audit and targeted regression tests
The author invented AES-GCM, HKDF, HMAC, ECDH, ECDSA, or Matrix E2EE	Rejected; these are established primitives and protocols
Every identified protection profile is an independently audited cipher	Rejected